

Outils et configuration pour le développement Web

Introduction

Avant de commencer à développer un site web ou une application, il est important de préparer correctement son environnement de travail.

Dans ce cours, nous allons apprendre à :

1. Installer et configurer un environnement de développement.
2. Comprendre comment fonctionne la communication entre un navigateur et un serveur.
3. Créer un repository GitHub.
4. Cloner un repository sur son ordinateur avec Visual Studio Code.
5. Installer les packages nécessaires à un projet Node.js.
6. Démarrer un serveur local avec Node.js.

À la fin de ce cours, vous serez capable de préparer un projet, récupérer son code depuis GitHub, installer ses dépendances et lancer un serveur local.

Chapitre 1 — Installer l'environnement de développement

1.1 Qu'est-ce qu'un environnement de développement ?

Un environnement de développement est l'ensemble des outils utilisés par un développeur pour créer, tester et modifier une application.

Pour le développement Web, on peut notamment avoir besoin de :

- **Visual Studio Code** pour écrire le code ;
 - **Node.js** pour exécuter du JavaScript côté serveur ;
 - **npm** pour installer les packages ;
 - **Git** pour gérer les versions du code ;
 - **GitHub** pour stocker et partager les repositories ;
 - un **navigateur Web** pour tester l'application.
-

1.2 Visual Studio Code

Visual Studio Code, souvent appelé VS Code, est un éditeur de code.

Il permet notamment de :

- créer des fichiers ;
- modifier du code ;
- organiser les dossiers d'un projet ;
- ouvrir un terminal ;
- utiliser Git ;
- installer des extensions ;
- travailler avec JavaScript, Node.js, HTML, CSS et de nombreux autres langages.

Un projet peut par exemple être ouvert dans VS Code avec une structure comme :

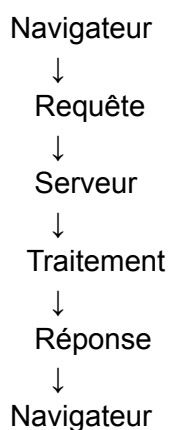
```
mon-projet/  
├── server.js  
├── package.json  
├── node_modules/  
└── README.md
```

Chapitre 2 — Comprendre les réponses du navigateur

2.1 Comment fonctionne une application Web ?

Lorsque vous ouvrez une adresse Web dans votre navigateur, celui-ci communique généralement avec un serveur.

Le fonctionnement simplifié est :



Par exemple, lorsque vous demandez :

`http://localhost:3000`

le navigateur envoie une requête au serveur qui fonctionne sur votre ordinateur.

Le serveur traite cette requête puis renvoie une réponse.

2.2 Les codes de réponse HTTP

Les réponses HTTP utilisent des codes permettant d'indiquer le résultat d'une requête.

200 — OK

Le code **200** indique généralement que la requête a réussi.

404 — Not Found

Le code **404** indique que la ressource demandée n'a pas été trouvée.

500 — Internal Server Error

Le code **500** indique généralement qu'une erreur s'est produite sur le serveur.

Il est donc important pour un développeur de savoir interpréter les réponses du navigateur lorsqu'il teste son application.

Chapitre 3 — Créer un repository GitHub

3.1 Qu'est-ce qu'un repository ?

Un **repository**, ou dépôt, est un espace permettant de stocker et de gérer le code d'un projet avec Git.

GitHub permet notamment de :

- sauvegarder le code ;
- suivre les modifications ;
- travailler en équipe ;
- partager un projet ;
- revenir à une ancienne version du code.

Un repository peut contenir :

```
mon-projet/  
├── fichiers du projet  
├── dossiers  
├── package.json  
└── README.md
```

3.2 Créer un repository

Pour créer un repository sur GitHub, on commence par se connecter à son compte GitHub.

Ensuite :

1. Créer un nouveau repository.
2. Donner un nom au repository.
3. Choisir sa visibilité.
4. Valider la création.

Par exemple :

Nom du repository :
automobiles

GitHub crée alors un espace dans lequel le projet pourra être enregistré.

Chapitre 4 — Cloner un repository sur son ordinateur

4.1 Qu'est-ce que cloner un repository ?

Cloner un repository signifie créer une copie du repository distant sur son ordinateur.

On peut alors modifier le projet localement.

Le principe est :

```
GitHub  
↓  
git clone  
↓  
Ordinateur
```

↓
Visual Studio Code

4.2 La commande git clone

Dans un terminal, on peut utiliser :

```
git clone URL_DU_REPOSITORY
```

Par exemple :

```
git clone https://github.com/utilisateur/automobiles.git
```

Git télécharge alors les fichiers du repository sur l'ordinateur.

4.3 Ouvrir le projet dans VS Code

Une fois le repository cloné, on peut entrer dans son dossier :

```
cd automobiles
```

Puis ouvrir le projet dans Visual Studio Code :

```
code .
```

Le projet est maintenant prêt à être utilisé dans VS Code.

Chapitre 5 — Node.js et les packages

5.1 Qu'est-ce que Node.js ?

Node.js permet d'exécuter du JavaScript en dehors du navigateur.

Il est notamment utilisé pour créer des serveurs et des applications Web.

Par exemple, un fichier :

```
server.js
```

peut contenir le code nécessaire au fonctionnement d'un serveur.

5.2 Qu'est-ce qu'un package ?

Un **package** est un ensemble de code réutilisable qu'un développeur peut ajouter à son projet.

Au lieu de programmer toutes les fonctionnalités soi-même, on peut utiliser des packages existants.

Node.js utilise généralement **npm** pour gérer ces packages.

Chapitre 6 — Installer les packages avec npm

6.1 Le fichier package.json

Un projet Node.js contient généralement un fichier :

package.json

Ce fichier contient notamment les informations du projet et ses dépendances.

Par exemple :

```
{  
  "name": "automobiles",  
  "version": "1.0.0"  
}
```

6.2 Installer les dépendances

Après avoir cloné un projet Node.js, il est généralement nécessaire d'installer ses dépendances.

On utilise :

npm install

Cette commande lit les dépendances du projet et installe les packages nécessaires.

Un dossier appelé :

node_modules

est généralement créé pour contenir les packages installés.

Chapitre 7 — Démarrer un serveur Node.js

Une fois les packages installés, on peut lancer le serveur.

Si le fichier principal s'appelle :

`server.js`

on peut utiliser :

`node server.js`

Si tout fonctionne correctement, le terminal peut afficher :

Serveur démarré sur `http://localhost:3000`

Cela signifie que le serveur fonctionne localement sur le **port 3000**.

Chapitre 8 — Comprendre localhost et le port

8.1 Qu'est-ce que localhost ?

`localhost` désigne l'ordinateur local sur lequel le programme est exécuté.

Ainsi :

`http://localhost:3000`

signifie que le navigateur doit communiquer avec un serveur exécuté sur l'ordinateur local.

8.2 Qu'est-ce que le port 3000 ?

Le **port** permet de distinguer différents services réseau exécutés sur une même machine.

Dans :

`http://localhost:3000`

on a :

`http://` → protocole
`localhost` → ordinateur local
`3000` → port

Lorsque le serveur écoute sur le port 3000, on peut ouvrir cette adresse dans le navigateur :

`http://localhost:3000`

Chapitre 9 — Workflow complet

Voici maintenant le processus complet que vous devez retenir.

Étape 1 — Créer le repository

Créer le repository du projet sur GitHub.

Étape 2 — Cloner le repository

`git clone URL_DU_REPOSITORY`

Étape 3 — Entrer dans le projet

`cd nom-du-projet`

Étape 4 — Ouvrir dans VS Code

`code .`

Étape 5 — Installer les packages

`npm install`

Étape 6 — Démarrer le serveur

`node server.js`

Étape 7 — Tester dans le navigateur

Ouvrir :

`http://localhost:3000`

Le workflow peut donc être résumé ainsi :

GitHub
↓
Cloner
↓
Ordinateur
↓
VS Code
↓
npm install
↓
node server.js
↓
localhost:3000
↓
Navigateur

Chapitre 10 — Points essentiels à retenir

Élément	Rôle
VS Code	Écrire et modifier le code
Git	Gérer les versions du projet
GitHub	Héberger et partager le repository
<code>git clone</code>	Copier un repository sur son ordinateur
Node.js	Exécuter JavaScript côté serveur
npm	Gérer les packages Node.js
<code>npm install</code>	Installer les dépendances
<code>node server.js</code>	Exécuter le serveur
localhost	Désigne l'ordinateur local
port 3000	Port utilisé par le serveur
navigateur	Permet de communiquer avec le serveur et d'afficher la réponse

Conclusion

La préparation de l'environnement est une étape fondamentale dans le développement Web.

Vous devez être capable de comprendre le chemin suivant :

Créer → cloner → ouvrir → installer → démarrer → tester.

Avec GitHub, vous pouvez récupérer le code d'un projet. Avec VS Code, vous pouvez le modifier. Avec npm, vous pouvez installer ses dépendances. Avec Node.js, vous pouvez démarrer le serveur. Enfin, avec le navigateur, vous pouvez tester l'application et observer les réponses du serveur.

Une fois ces notions maîtrisées, vous disposez des bases nécessaires pour commencer à travailler efficacement sur des projets Web modernes.